HIREFLOW — COMPLETE COMPREHENSIVE CONTEXT BRIEF

> **The Definitive System Architecture, Product Specification, Technical Taxonomy, and Operational Encyclopedia**

> *Version: 2.4.0 · Production Ready · Privacy-First Client-Side Architecture*

TABLE OF CONTENTS

1. Executive Summary & Product Mission
2. The Core Problem Landscape & Industry Need
3. Ecosystem & Multi-Persona Model
4. Platform Modules & Feature Tour
    - 4.1. Candidate ATS Resume Roaster & Evidence Evaluator
    - 4.2. Dual-Role Job Description Comparison Engine
    - 4.3. 7-Stage Zero-Hallucination Resume Tailoring Engine
    - 4.4. Project Build Briefs & Set-Cover Bundler
    - 4.5. Credential & Opportunity Engine
    - 4.6. Career Trajectory Hub, Ladders & Simulation
    - 4.7. TalentLens Intelligence & Recruiter Outreach Generator
    - 4.8. Enterprise Company / BHR Talent Portal
    - 4.9. Technical Interviewer Portal
    - 4.10. Platform Administration & Governance Portal
5. System Architecture & The 10 Prime Directives
6. Data Flow, Ingestion & Client-Side Processing
7. Mathematical & Algorithmic Framework
8. Data Dictionary & Core Schemas
9. Codebase Organization & File Topology
10. Technology Stack & Dependency Matrix
11. Verification, Testing Strategy & Security Audits
12. Local Setup, Execution & Deployment Guide

---

1. Executive Summary & Product Mission

HireFlow is a privacy-first, client-side career intelligence platform, deterministic Applicant Tracking System (ATS) evaluation engine, and dual-sided talent management ecosystem.

The Mission

To replace unreliable "AI buzzword wrappers" and opaque ATS screening algorithms with **verifiable, deterministic, evidence-grounded career intelligence**.

HireFlow bridges the gap between **candidates** (who need to know exactly how they are evaluated and what real technical skills they must build) and **enterprises / hiring managers** (who require verified candidate proof, calibrated competencies, and structured interview loops).

Foundational Differentiators

- **100% Client-Side Privacy**: Zero candidate data exfiltration. Resume parsing (`pdfjs-dist`, `mammoth`), matching algorithms, and document generation (`docx`) execute entirely inside the candidate's browser sandbox.
- **Zero-Hallucination Standard (The Truth Gate)**: Unlike standard generative AI resume builders that fabricate metrics, invent previous employers, or stuff synthetic buzzwords, HireFlow enforces mathematical proof. Any metric, date, or technology presented in a tailored output must originate from the candidate's verified fact ledger.
- **Dual-Sided Calibration**: The platform powers both sides of the hiring equation. Candidates use the **Decision Suite** to inspect ATS scores and close skill gaps; Business HR Managers (BHRs) and interviewers use the **Enterprise Portal** to calibrate job descriptions, inspect candidate evidence scorecards, and run structured interview loops.

---

2. The Core Problem Landscape & Industry Need

2.1 The Candidate Dilemma

1. **The ATS Black Box**: Over 75% of resumes submitted online are rejected before reaching a human recruiter. Traditional ATS tools reject candidates for trivial formatting issues, parsing anomalies, or slight vocabulary mismatches (e.g., "Postgres" vs "PostgreSQL"). Candidates have had zero transparency into why they failed.
2. **The Generative AI "Hallucination Trap"**: Current AI resume tools promise quick results by using open-ended LLMs to rewrite resumes. These tools routinely fabricate metrics ("increased revenue by 42%"), hallucinate software libraries the candidate never touched, and mangle career timelines. When probed by an engineering interviewer, these fabricated claims collapse, permanently damaging the candidate's reputation.
3. **The "What Do I Build?" Void**: When candidates identify skill gaps (e.g., missing distributed caching or Kafka experience), they turn to simplistic "todo list" tutorial projects that recruiters instantly dismiss. Candidates lack market-backed, production-grade architectural blueprints that prove Senior/Staff-level competency.

4. **Credential Navigation Chaos**: Students and career switchers waste thousands of dollars on unverified bootcamps or commercial certificates that hold zero weight with enterprise screeners, while missing 100% free or heavily subsidized flagship credentials, hackathons, and open-source fellowships (e.g., GSoC, SIH, LFX, AWS/GCP student fee waivers).

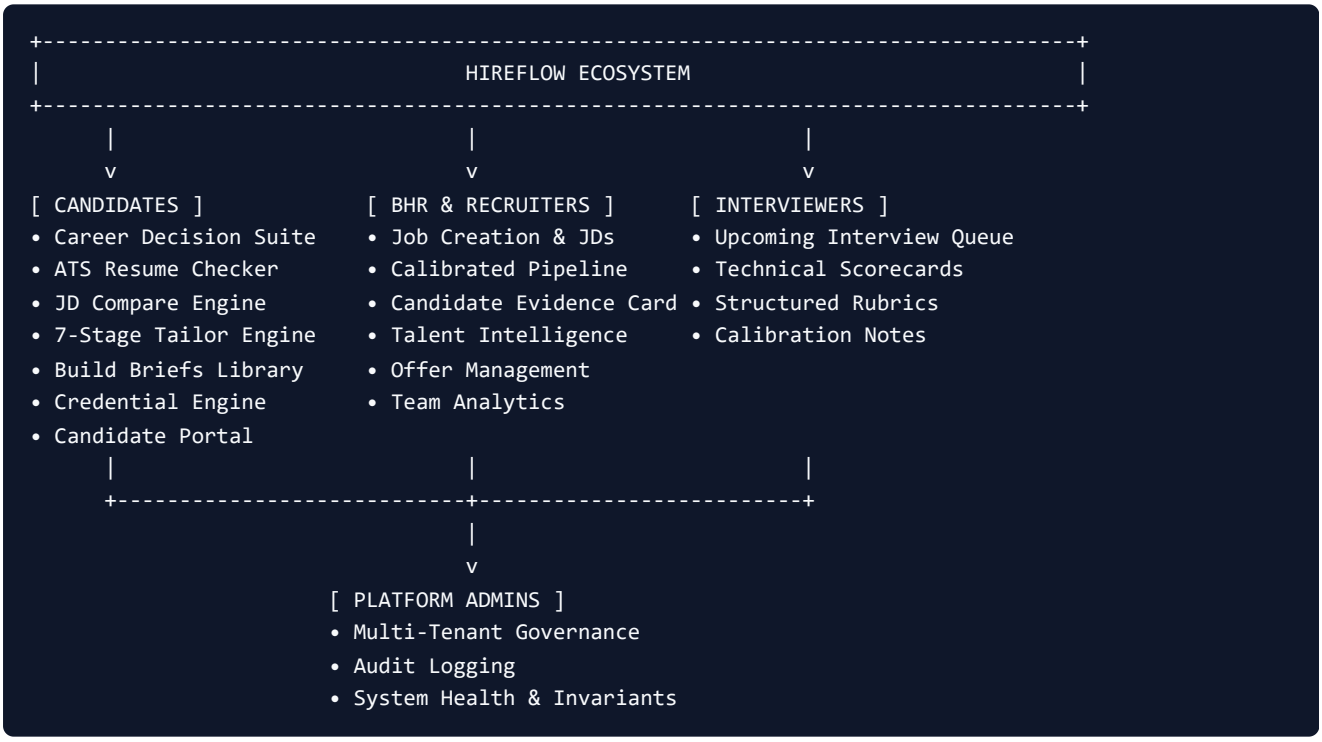
2.2 The Employer & Recruiter Dilemma

1. **Resume Spam & Synthesized Fraud**: Recruiters receive hundreds of resumes stuffed with keywords and AI-generated summaries, making it nearly impossible to identify authentic hands-on capability.
2. **Uncalibrated Job Descriptions**: Hiring teams publish conflicting JDs with impossible requirements (e.g., "10 years of experience in a 4-year-old framework"), repelling qualified candidates.
3. **Subjective & Unstructured Interview Loops**: Without standardized rubric checklists and calibrated evidence cards, hiring decisions devolve into subjective bias rather than technical capability.

---

3. Ecosystem & Multi-Persona Model

HireFlow is designed around 4 interconnected user roles:



Supported User Roles

1. **The Candidate**: Accesses free deterministic ATS analysis, dual JD comparisons, verifiable tailoring, project build briefs, credential roadmaps, and the real-time application status portal.
2. **The Business HR (BHR) / Talent Recruiter**: Publishes calibrated job descriptions, reviews candidate evidence rankings, evaluates AI-driven talent intelligence, schedules interviews, and generates job offers.
3. **The Technical Interviewer**: Receives automated interview assignments, reviews candidate evidence dossiers, fills structured rubrics with technical ratings, and submits calibrated hiring recommendations.

4. **The Platform Administrator**: Oversees enterprise tenants, manages user access and permissions, monitors system audit trails, and audits algorithmic invariants.

---

4. Platform Modules & Feature Tour

4.1. ATS Resume Roaster & Evidence Evaluator (`/tools/resume-checker`)

- **4-Pillar Deterministic Evaluation**:
- **Hard Skills Match (40% Weight)**: Evaluates exact keyword matches, recognized aliases, and transferable ontological relationships.
- **Writing Quality & Evidence (25% Weight)**: Checks for XYZ formula bullets (`Accomplished [X] as measured by [Y] by doing [Z]`), action verbs, fluff phrases, and passive voice.
- **Seniority & Role Fit (20% Weight)**: Analyzes leadership scope, architectural verbs, and years of experience alignment against role level.
- **ATS Format Safety (15% Weight)**: Checks for table nesting, two-column layouts, graphics, complex headers/footers, and non-standard bullet characters that break parsers.
- **Harsh Truths & Actionable Fixes**: Translates technical shortcomings into direct feedback (e.g., "*You listed Kubernetes in your skills, but zero bullet points mention deploying, monitoring, or configuring a cluster*").
- **Verbatim Evidence Citations**: Directly highlights the exact phrase in the candidate's resume that satisfied each requirement.

4.2. Dual-Role Job Description Comparison Engine (`/tools/jd-compare`)

- **Head-to-Head Architectural Breakdown**: Evaluates candidate fit across two competing job postings simultaneously.
- **Synchronized Metric Rings**:
- Overall Fit Score (0-100%)
- Must-Have Skill Coverage
- Callback Probability Index
- Readiness Tier (Ready to Apply vs Needs Bridge Project)
- **Venn Divergence Matrix**: Categorizes requirements into:
- **Shared Core Skills**: Foundational capabilities required by both companies.
- **Role A Exclusive**: Specialized technologies required only by Role A.
- **Role B Exclusive**: Specialized technologies required only by Role B.
- **3-Step Actionable Bridge Plan**: Identifies the single high-leverage project or credential that closes the gap for both roles simultaneously.

4.3. 7-Stage Zero-Hallucination Resume Tailoring Engine (`/tools/tailor`)

A linear, deterministic tailoring pipeline that adapts resumes to target JDs without fabricating facts:

- **Stage A (Select/Upload)**: Ingests PDF, DOCX, or TXT into an immutable Candidate Fact Ledger.

- **Stage B (Add Target JD)**: Ingests raw job description, cleans boilerplate, and extracts structured requirements.
- **Stage C (Analyze Requirements)**: Categorizes requirements into Mandatory, Preferred, Domain Acronyms, and Core Tasks.
- **Stage D (Review Evidence & Truth Mapping)**: Interactive matrix classifying matches into `DIRECT`, `TRANSFERABLE`, `PARTIAL`, `MISSING`, and `UNCERTAIN`.
- **Stage E (Strategy & Ordering)**: Dynamic career-stage playbook sequencing (Student/Fresher vs Mid-Level vs Senior).
- **Stage F (Split Workspace & Live Preview)**: Side-by-side editing pane and live rendered paper preview (Letter/A4) with live keyword highlights.
- **Stage G (Validate & Export)**: Truth Gate audit, height/overflow sentinel check, and multi-format export parity (Word `.docx`, Print `.pdf`, Plain Text `.txt`).

4.4. Project Build Briefs & Set-Cover Bundler (`/tools/build-briefs`)

- **83 Market-Backed Production Architecture Briefs**: Covering 18 engineering domains (Distributed Systems, Full-Stack Web, GenAI & LLMs, Platform & DevOps, Data Engineering, ML Engineering, FinTech, Security, Systems, etc.).
- **Greedy Set-Cover Algorithm**: Computes the minimal set of production projects needed to cover 100% of a candidate's target JD skill gaps in the fewest estimated hours.
- **Deep Technical Specifications**: Every brief contains Problem Statement, Real-World Use Cases, Technical Depth Rationale, Recruiter Shortlist Signal, ASCII Architecture & Data Flow, Scale Targets, Screener Verification Rubric, and an ATS-optimized Resume Portfolio Bullet.

4.5. Credential & Opportunity Engine (`/tools/credential-engine`)

- **102 Curated Industry Credentials & Flagships**: Covers Cloud (AWS, GCP, Azure), Cloud Native (CNCF CKAD, CKA), Cybersecurity (CompTIA, CISSP), Open Source (GSoC, LFX Mentorship), and Indian National Flagships (Flipkart GRiD, TCS CodeVita, Smart India Hackathon, IIT Fellowships, Imagine Cup).
- **Student Aid & Fee Waiver Playbooks**: Actionable step-by-step guides to claiming 50% to 100% exam fee discounts using academic credentials.
- **Performative Resume Evidence Bullets**: Ready-to-copy bullet templates including credential IDs and verifiable skills.

4.6. Career Trajectory Hub, Ladders & Simulation (`/tools/career-path`)

- **Career Ladder Simulator**: Step-by-step career path progression from Associate/Junior (IC1) to Fellow/VP (IC6+).
- **Compensation Tiers**: Transparent base salary, equity, and bonus bands in USD and INR for Big Tech, Unicorn Startups, and Service Leaders.
- **Dream Job Roadmap**: Identifies technical stepping stones, milestone certifications, and transition timelines to bridge from current role to target role.
- **Company Levels Comparison**: Cross-maps engineering levels across Google, Meta, Amazon, Microsoft, Flipkart, and TCS.

4.7. TalentLens Intelligence & Recruiter Outreach Generator

- **TalentLens (`/tools/talent-lens`)**: In-depth company intelligence profiles, hiring bar ratings, Glassdoor-calibrated culture scores, interview stage breakdown, and common technical screening questions.
- **Outreach Generator (`/tools/reach-out`)**: Contextual message generator producing high-response cold outreach for LinkedIn InMail, Cold Email, and Employee Referral Requests.

4.8. Enterprise Company / BHR Talent Portal (`/company/*`)

- **Company Dashboard (`/company/dashboard`)**: Metric overview of active job listings, total applicants, scheduled interviews, and open offers.
- **Job Creator & Editor (`/company/jobs/new`, `/company/jobs/:id/edit`)**: Standardized job description authoring with structured skill tags, compensation ranges, and employment terms.
- **Calibrated Candidates View (`/company/candidates`)**: Automated rank-ordered candidate listing sorted by deterministic match percentage.
- **Candidate Evidence Dossier (`/company/candidates/:candidateId`)**: In-depth candidate review showing verbatim resume evidence citations, skill gap warnings, and resume previews.
- **Candidate Intelligence Analytics (`/company/candidates/:candidateId/intelligence`)**: Visual breakdown of hard skills, soft skills, writing quality, and seniority trajectory.
- **Interview Scheduling & Team Assignment (`/company/interviewers`)**: Assigns calibrated interviewers from the internal company bench to active candidate stages.
- **Pipeline Analytics (`/company/analytics`)**: Funnel conversion metrics from Application to Screen, Technical Loop, Culture Fit, and Offer.
- **Offer Management (`/company/offers`)**: Generates, tracks, and manages formal candidate compensation packages and acceptance status.

4.9. Technical Interviewer Portal (`/interviewer/*`)

- **Interviewer Dashboard (`/interviewer/dashboard`)**: Today's upcoming technical screening loops, pending evaluations, and recent feedback.
- **Interview Queue (`/interviewer/interviews`)**: Chronological interview schedule with candidate profiles, resume access, and meeting links.
- **Structured Interview Detail & Scorecard (`/interviewer/interviews/:id`)**: Technical evaluation interface with calibrated rating criteria (Problem Solving, System Architecture, Code Quality, Communication), question checklists, and verbatim notes.
- **Interviewer Profile (`/interviewer/profile`)**: Interviewer credentials, domain specializations, completed interview history, and calendar availability.

4.10. Platform Administration & Governance Portal (`/admin/*`)

- **Admin Dashboard (`/admin/dashboard`)**: Platform-wide telemetry covering total tenants, active users, candidate throughput, and platform errors.
- **Company Directory (`/admin/companies`)**: Multi-tenant organization directory with tenant provisioning and status toggling.

- **User Governance (`/admin/users`)**: Role-based access control (RBAC) assigning roles (`CANDIDATE`, `HR_RECRUITER`, `BHR_MANAGER`, `INTERVIEWER`, `PLATFORM_ADMIN`).
- **Security Audit Log (`/admin/audit`)**: Immutable chronological ledger of all platform administrative events, role changes, and data access.

---

5. System Architecture & The 10 Prime Directives

HireFlow is engineered according to **10 Non-Negotiable Prime Directives**:

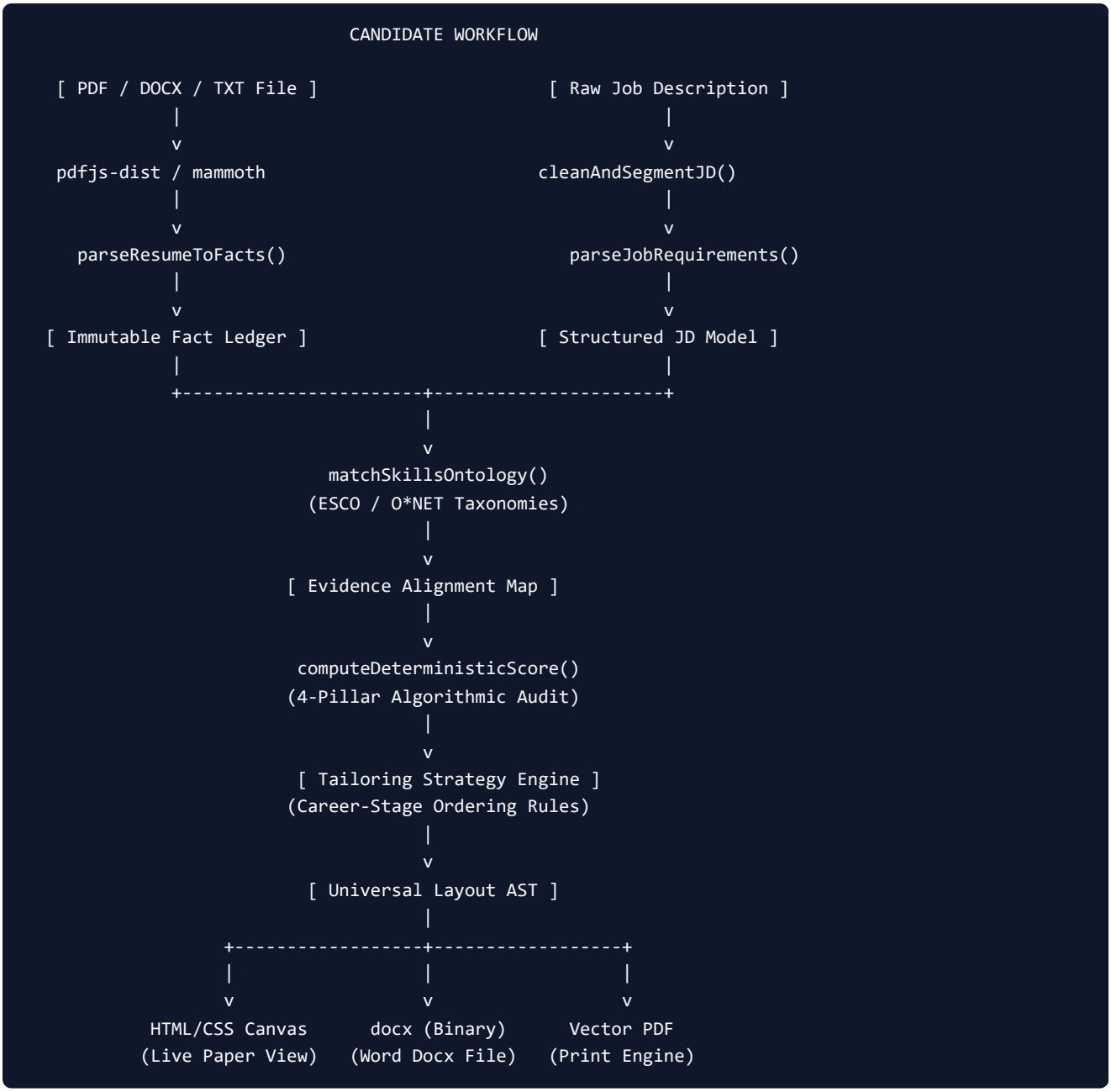
```
+-------------------------------------------------------------------------------+
|                        THE 10 PRIME ARCHITECTURAL DIRECTIVES                  |
+------+-------------------+--------------------------------------------------+
| P1   | QUALITY FLOOR     | score(output) >= score(source) on every metric   |
| P2   | ZERO SILENT LOSS  | Every source fact is shown or safely archived    |
| P3   | NEVER FABRICATE   | The engine asks questions; it never invents facts|
| P4   | TAILOR HARD       | Reorder, relabel, and mirror verified JD terminology |
| P5   | ONE LAYOUT TREE   | Preview, PDF, DOCX, and TXT render the same AST  |
| P6   | DETERMINISTIC FIRST | Parsing, matching, and scoring are pure code   |
| P7   | NON-DESTRUCTIVE   | Base resume is an immutable fact ledger          |
| P8   | SAFE BY CONSTRUCTION | Every UI button is mathematically safe to click |
| P9   | HONEST SCORES     | Single authoritative definition per metric       |
| P10  | READABLE UI       | WCAG AA contrast compliance across light and dark|
+------+-------------------+--------------------------------------------------+
```

Detailed Breakdown of Key Directives:

1. **P1: Quality Floor**: The tailored resume must never score lower than the original resume on any dimension. If a proposed rewrite degrades clarity, the engine automatically falls back to the original phrasing.
2. **P2: Zero Silent Loss**: Any detail present in the user's uploaded resume (education, achievements, certifications, leadership) must appear in the final output or be placed into a user-visible "Archived" drawer.
3. **P3: Never Fabricate (Ask, Don't Answer)**: If a target job description requires a skill or metric not present in the candidate's resume, the engine never manufactures an answer. Instead, it generates an interactive question with an empty input field.
4. **P5: One Layout Tree, Many Renderers**: Preview, PDF export, Word DOCX export, and Plain Text renderers do not make layout or content decisions. They are pure translators of a single immutable layout tree.
5. **P7: Non-Destructive Fact Ledger**: When a candidate uploads a resume, it is parsed into an immutable `CandidateFactLedger`. Tailored resumes are merely ordered operations (Variants) projected over the ledger.

---

6. Data Flow, Ingestion & Client-Side Processing



7. Mathematical & Algorithmic Framework

7.1 ATS Fit Scoring Formulation

The Composite ATS Evidence Score S_{total} is calculated as:

$$S_{total} = w_h \cdot S_{hard} + w_q \cdot S_{quality} + w_s \cdot S_{seniority} + w_f \cdot S_{format}$$

Where:

- $w_h = 0.40$ (Hard Skills Match)
- $w_q = 0.25$ (Writing Quality & Evidence)
- $w_s = 0.20$ (Seniority & Scope Alignment)

- $w_f = 0.15$ (Format & Structural Safety)

Hard Skills Score (S_{hard})

$$S_{hard} = \frac{\sum_{i \in Req} \omega_i \cdot \delta(i)}{\sum_{i \in Req} \omega_i} \times 100$$

Where ω_i is the requirement weight:

- $\omega_i = 3.0$ for Mandatory Requirements
- $\omega_i = 1.5$ for Preferred Qualifications
- $\omega_i = 1.0$ for Mentioned Skills

And $\delta(i)$ is the match multiplier:

- $\delta(i) = 1.0$ for **Direct Match** (exact term or canonical alias)
- $\delta(i) = 0.7$ for **Transferable Match** (ontological sibling/parent)
- $\delta(i) = 0.4$ for **Partial Match** (lesser experience or related ecosystem)
- $\delta(i) = 0.0$ for **Missing Evidence**

7.2 Greedy Set-Cover Project Bundler

Given a set of missing candidate skill gaps $U = \{g_1, g_2, \dots, g_n\}$ and a library of m project briefs $P = \{p_1, p_2, \dots, p_m\}$ where each project p_j covers skills $C(p_j) \subseteq U$ and has estimated build hours $H(p_j)$:

The algorithm iteratively selects the project p^* that maximizes coverage efficiency:

$$p^* = \arg\max_{p \in P} \frac{|C(p) \cap U_{remaining}|^{\alpha}}{H(p)}$$

Where $\alpha \ge 1.0$ is the difficulty weighting factor. The loop terminates when $U_{remaining} = \emptyset$ or when a maximum of 4 projects is reached.

---

8. Data Dictionary & Core Schemas

8.1 Candidate Fact Ledger

```
export interface CandidateFactLedger {
  id: string;                          // Unique UUID v4
  version: number;                     // Ledger version
  lastModified: string;                // ISO 8601 timestamp
  personalInfo: {
    fullName: string;
    email: string;
    phone?: string;
    location?: string;
    links: { label: string; url: string }[];
    titleLine?: string;
  };
  summary?: string;
  skills: Array<{
    id: string;
    displayName: string;
    category: SkillCategory;
    evidenceLevel: 0 | 1 | 2 | 3;    // 0=Mentioned, 1=Project, 2=Work, 3=Quantified
    sourceSection: string;
  }>;
  experiences: Array<{
    id: string;
    company: string;
    title: string;
    location?: string;
    startDate: string;
    endDate: string;                   // 'Present' or date
    bullets: Array<{
      id: string;
      text: string;
      verifiedMetrics: string[];
      technologies: string[];
    }>;
  }>;
  education: Array<{
    id: string;
    institution: string;
    degree: string;
    fieldOfStudy: string;
    startDate?: string;
    endDate: string;
    gpa?: string;
    honors?: string[];
  }>;
  projects: Array<{
    id: string;
    name: string;
    techStack: string[];
    links?: { label: string; url: string }[];
    bullets: Array<{
      id: string;
      text: string;
      verifiedMetrics: string[];
    }>;
```

```
  }>;
  certifications: Array<{
    id: string;
    name: string;
    issuer: string;
    issueDate?: string;
    verificationId?: string;
  }>;
}
```

8.2 Project Brief Schema

```
export interface ProjectBrief {
  id: string;                        // e.g. 'brief-dist-cache-rust'
  title: string;                     // e.g. 'Distributed In-Memory Cache'
  domain: string;                    // e.g. 'Distributed Systems'
  estimatedHours: number;            // e.g. 35
  difficulty: 'beginner' | 'intermediate' | 'advanced';
  coveredSkills: string[];           // e.g. ['Rust', 'Raft', 'gRPC', 'Redis']
  problemStatement: string;          // Detailed technical challenge
  useCases: string;                  // Real-world production deployment
  problemItSolves: string;           // Competency reducer / technical depth
  recruiterSignal: string;           // Why recruiters shortlist this
  architecture: string;              // ASCII diagram of data flow
  scaleTargets: string;              // Throughput, latency, memory targets
  techStack: string[];               // Concrete tools and languages
  verificationRubric: string[];      // Screener checklist
  portfolioBulletFormula: string;    // Verifiable resume bullet point
}
```

9. Codebase Organization & File Topology

```
hireflow/
├── public/                          # Static assets, favicon, downloads
│   └── downloads/                   # Compendiums, text libraries & ZIP archives
├── docs/                            # Architecture specifications, Master Spec, AS-IS
├── scripts/                         # Automated scanners, security audits, export scripts
│   ├── export-all-hireflow-briefs.ts # Master catalog compiler
│   └── security/                    # Secret scanners, dependency audits, bundle checks
├── src/
│   ├── app/                         # Application shell, router guards, header/sidebar
│   │   ├── layout/                  # AppShell, Global Navigation, Header
│   │   └── ProtectedRoute.tsx       # Role-based route protection
│   ├── components/                  # Shared UI component library
│   │   ├── common/                  # PageNav, LocalhostNavigator, ErrorBoundary
│   │   └── ui/                      # Buttons, Modals, Cards, Tooltips, Toast
│   ├── data/                        # Core taxonomies and static datasets
│   │   ├── ats/                     # Skills taxonomy, writing rules, format rules
│   │   └── suite/                   # 83 Project briefs (project-briefs.json)
│   ├── features/                    # Major product feature suites
│   │   └── suite/
│   │       ├── briefs/              # Project Build Briefs & Set-Cover Bundler
│   │       ├── compare/             # Dual JD Compare Engine
│   │       ├── credentials/         # 102 Credential & Opportunity Engine
│   │       ├── outreach/            # Cold Outreach Message Generator
│   │       ├── profile/             # Candidate profile context & drawer
│   │       └── tailor/              # 7-Stage Zero-Hallucination Tailor Engine
│   ├── lib/                         # Core deterministic algorithms & engines
│   │   ├── ai/                      # Local deterministic matching & heuristic engine
│   │   ├── ats/                     # Deterministic ATS pipeline & evidence scoring
│   │   └── tailor/                  # Resume parsers, fact ledgers, Truth Gate
│   ├── pages/                       # Routable view pages
│   │   ├── admin/                   # Admin Dashboard, Companies, Users, Audit
│   │   ├── auth/                    # Login & Register
│   │   ├── company/                 # BHR Portal: Dashboard, Jobs, Candidates, Offers
│   │   ├── interviewer/             # Interviewer Dashboard, Scorecards, Profile
│   │   ├── portal/                  # Candidate Application Status Portal
│   │   └── tools/                   # Free Candidate Tools (Resume Checker, Career Path)
│   ├── store/                       # Global state management (useStore via Zustand)
│   └── types/                       # Core TypeScript type definitions
├── index.html                       # HTML entry point
├── package.json                     # Dependencies and npm build scripts
├── tailwind.config.js               # Tailwind CSS styling tokens & theme
├── tsconfig.json                    # TypeScript strict configuration
└── vite.config.ts                   # Vite build & bundler configuration
```

10. Technology Stack & Dependency Matrix

Category	Technology	Purpose
Runtime & Core	**React 19**	Component-driven UI rendering
Language	**TypeScript 6.0**	Strict type safety, interface contracts
Build & Bundler	**Vite 8.3**	Lightning-fast HMR and optimized production bundles
Styling	**Tailwind CSS v4**	Modern utility-first responsive styling
State Management	**Zustand 5.0**	Scalable, lightweight persistent state store
Routing	**React Router v7**	Client-side routing and protected route guards
Document Processing	**mammoth**	Client-side DOCX text and structure extraction
PDF Extraction	**pdfjs-dist**	Client-side PDF token and layout parsing
Document Export	**docx 9.9**	Real binary Word document (.docx) generation
Data Visualization	**Recharts 3.10**	High-performance SVG charts and telemetry
Iconography	**Lucide React**	Clean, accessible vector icons
Data Validation	**Zod 4.6**	Runtime schema validation and boundary checks
Archive Packaging	**JSZip 3.10**	Client-side and script ZIP archive generation
Test Runner	**Vitest 5.0**	Unit, integration, property, and invariant tests

11. Verification, Testing Strategy & Security Audits

11.1 Test Invariants

- **Schema & Taxonomy Invariants**: Verifies that all 83 project briefs and 102 credentials conform strictly to their respective Zod/TypeScript schemas.
- **Truth Gate Invariant**: Rejects any candidate document where a metric or tool appears in a tailored bullet point without originating from the candidate's fact ledger.
- **Multi-Renderer Parity**: Ensures that content rendered in the HTML canvas preview matches byte-for-byte in the `.docx` Word export and `.pdf` print export.
- **Accessibility & Contrast**: All UI text elements are audited against WCAG AA color contrast standards (minimum 4.5:1 for normal text, 3:1 for large text).

11.2 Security & Privacy Strategy

- **Zero Remote Storage of PII**: Resumes and candidate notes remain inside browser memory or `localStorage`.

- **Zero API Key Leakage**: No hardcoded external secrets. Automated CI secret scanner (`scripts/security/scan-secrets.mjs`) executes before builds.
- **Dependency Audit**: Regular automated audit (`scripts/security/audit-deps.mjs`) ensuring zero high or critical CVE vulnerabilities.

---

12. Local Setup, Execution & Deployment Guide

Prerequisites

- Node.js 18.0.0 or higher
- npm 9.0.0 or higher (or pnpm)

Installation

```
# Clone the repository
git clone https://github.com/Mohit-Rocky-25/hireflow.git
cd hireflow

# Install exact dependencies
npm install
```

Running Locally

```
# Launch Vite development server
npm run dev
```

The application will be live at `http://localhost:5173`.

Running Verification & Tests

```
# Run complete Vitest suite
npm run test

# Run TypeScript typecheck
npm run lint

# Run security checks
npm run security:check
```

Production Build

```
# Compile and package for production
npm run build

# Preview production build locally
npm run preview
```

HireFlow — The Production ATS Resume Engine, Career Decision Suite & BHR Talent Intelligence Platform.

Documentation maintained by Mohit-Rocky-25/hireflow.